

Send and retrieve events from Azure Event Hubs

In this exercise, you create Azure Event Hubs resources and build a .NET console app to send and receive events using the **Azure.Messaging.EventHubs** SDK. You learn how to provision cloud resources, interact with Event Hubs, and clean up your environment when finished.

Tasks performed in this exercise:

- Create a resource group
- Create Azure Event Hubs resources
- Create a .NET console app to send and retrieve events
- Clean up resources

This exercise takes approximately **30** minutes to complete.

Create Azure Event Hubs resources

In this section of the exercise you create the needed resources in Azure with the Azure CLI.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Use the [**>**_] button to the right of the search bar at the top of the page to create a new cloud shell in the Azure portal, selecting a **Bash** environment. The cloud shell provides a command line interface in a pane at the bottom of the Azure portal. If you are prompted to select a storage account to persist your files, select **No storage account required**, your subscription, and then select **Apply**.

Note: If you have previously created a cloud shell that uses a *PowerShell* environment, switch it to **Bash**.

3. In the cloud shell toolbar, in the **Settings** menu, select **Go to Classic version** (this is required to use the code editor).
4. Create a resource group for the resources needed for this exercise. If you already have a resource group you want to use, proceed to the next step. Replace **myResourceGroup** with a name you want to use for the resource group. You can replace **eastus** with a region near you if needed.

In this **Cloudslice** lab, your **Resource Group** has already been created (**myResourceGrouplod58046262**) and you may safely skip this step.

TypeCopy

```
az group create --name myResourceGroup --location eastus
```

5. Many of the commands require unique names and use the same parameters. Creating some variables will reduce the changes needed to the commands that create resources. Run the following commands to create the needed variables. Replace **myResourceGroup** with the name you're using for this exercise. If you changed the location in the previous step, make the same change in the **location** variable.

TypeCopy

```
resourceGroup=myResourceGroupplod58046262
```

```
location=eastus
```

```
namespaceName=eventhubsns58046262
```

Create an Azure Event Hubs namespace and event hub

An Azure Event Hubs namespace is a logical container for event hub resources within Azure. It provides a unique scoping container where you can create one or more event hubs, which are used to ingest, process, and store large volumes of event data. The following instructions are performed in the cloud shell.

1. Run the following command to create an Event Hubs namespace.

TypeCopy

```
az eventhubs namespace create --name $namespaceName --resource-group $resourceGroup -l $location
```

2. Run the following command to create an event hub named **myEventHub** in the Event Hubs namespace.

TypeCopy

```
az eventhubs eventhub create --name myEventHub --resource-group $resourceGroup \
--namespace-name $namespaceName
```

Assign a role to your Microsoft Entra user name

To allow your app to send and receive messages, assign your Microsoft Entra user to the **Azure Event Hubs Data Owner** role at the Event Hubs namespace level. This gives your user account permission to manage and access queues and topics using Azure RBAC. Perform the following steps in the cloud shell.

1. Run the following command to retrieve the **userPrincipalName** from your account. This represents who the role will be assigned to.

TypeCopy

```
userPrincipal=$(az rest --method GET --url https://graph.microsoft.com/v1.0/me \
--headers 'Content-Type=application/json' \
--query userPrincipalName --output tsv)
```

2. Run the following command to retrieve the resource ID of the Event Hubs namespace. The resource ID sets the scope for the role assignment to a specific namespace.

TypeCopy

```
resourceID=$(az eventhubs namespace show --resource-group $resourceGroup \
--name $namespaceName --query id --output tsv)
```

3. Run the following command to create and assign the **Azure Event Hubs Data Owner** role, which gives you permission to send and retrieve events.

TypeCopy

```
az role assignment create --assignee $userPrincipal \
--role "Azure Event Hubs Data Owner" \
--scope $resourceID
```

Send and retrieve events with a .NET console application

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in the cloud shell.

Tip: Resize the cloud shell to display more information, and code, by dragging the top border. You can also use the minimize and maximize buttons to switch between the cloud shell and the main portal interface.

1. Run the following commands to create a directory to contain the project and change into the project directory.

TypeCopy

```
mkdir eventhubs
cd eventhubs
```

2. Create the .NET console application.

TypeCopy

```
dotnet new console
```

3. Run the following commands to add the **Azure.Messaging.EventHubs** and **Azure.Identity** packages to the project.

TypeCopy

```
dotnet add package Azure.Messaging.EventHubs
```

```
dotnet add package Azure.Identity
```

Now it's time to replace the template code in the **Program.cs** file using the editor in the cloud shell.

Add the starter code for the project

1. Run the following command in the cloud shell to begin editing the application.

TypeCopy

code Program.cs

2. Replace any existing contents with the following code. Be sure to review the comments in the code, and replace **YOUR_EVENT_HUB_NAMESPACE** with your event hub namespace.

csharpTypeCopy

```
using Azure.Messaging.EventHubs;
```

```
using Azure.Messaging.EventHubs.Producer;
```

```
using Azure.Messaging.EventHubs.Consumer;
```

```
using Azure.Identity;
```

```
using System.Text;
```

```
// TO-DO: Replace YOUR_EVENT_HUB_NAMESPACE with your actual Event Hub namespace
```

```
string namespaceURL = "YOUR_EVENT_HUB_NAMESPACE.servicebus.windows.net";
```

```
string eventHubName = "myEventHub";
```

```
// Create a DefaultAzureCredentialOptions object to exclude certain credentials
```

```
DefaultAzureCredentialOptions options = new()
```

```
{
```

```

        ExcludeEnvironmentCredential = true,
        ExcludeManagedIdentityCredential = true
    };

    // Number of events to be sent to the event hub

    int numOfEvents = 3;

    // CREATE A PRODUCER CLIENT AND SEND EVENTS

```

```

// CREATE A CONSUMER CLIENT AND RECEIVE EVENTS

```

3. Press **ctrl+s** to save your changes.

Add code to complete the application

In this section you add code to create the producer and consumer clients to send and receive events.

1. Locate the **// CREATE A PRODUCER CLIENT AND SEND EVENTS** comment and add the following code directly after the comment. Be sure to review the comments in the code.

csharpTypeCopy

```

// Create a producer client to send events to the event hub

```

```

EventHubProducerClient producerClient = new EventHubProducerClient(
    namespaceURL,
    eventHubName,
    new DefaultAzureCredential(options));

```

```

// Create a batch of events

```

```

using EventDataBatch eventBatch = await producerClient.CreateBatchAsync();

```

```

// Adding a random number to the event body and sending the events.
var random = new Random();
for (int i = 1; i <= numOfEvents; i++)
{
    int randomNumber = random.Next(1, 101); // 1 to 100 inclusive
    string eventBody = $"Event {randomNumber}";
    if (!eventBatch.TryAdd(new EventData(Encoding.UTF8.GetBytes(eventBody))))
    {
        // if it is too large for the batch
        throw new Exception($"Event {i} is too large for the batch and cannot be sent.");
    }
}

try
{
    // Use the producer client to send the batch of events to the event hub
    await producerClient.SendAsync(eventBatch);

    Console.WriteLine($"A batch of {numOfEvents} events has been published.");
    Console.WriteLine("Press Enter to retrieve and print the events...");
    Console.ReadLine();
}
finally
{
    await producerClient.DisposeAsync();
}

```

2. Press **ctrl+s** to save your changes.

3. Locate the **// CREATE A CONSUMER CLIENT AND RETRIEVE EVENTS** comment and add the following code directly after the comment. Be sure to review the comments in the code.

```
csharpTypeCopy
```

```
// Create an EventHubConsumerClient
```

```
await using var consumerClient = new EventHubConsumerClient(  
    EventHubConsumerClient.DefaultConsumerGroupName,  
    namespaceURL,  
    eventHubName,  
    new DefaultAzureCredential(options));
```

```
Console.Clear();
```

```
Console.WriteLine("Retrieving all events from the hub...");
```

```
// Get total number of events in the hub by summing (last - first + 1) for all partitions
```

```
// This count is used to determine when to stop reading events
```

```
long totalEventCount = 0;
```

```
string[] partitionIds = await consumerClient.GetPartitionIdsAsync();
```

```
foreach (var partitionId in partitionIds)
```

```
{
```

```
    PartitionProperties properties = await  
    consumerClient.GetPartitionPropertiesAsync(partitionId);
```

```
    if (!properties.IsEmpty && properties.LastEnqueuedSequenceNumber >=  
        properties.BeginningSequenceNumber)
```

```
    {
```

```
        totalEventCount += (properties.LastEnqueuedSequenceNumber -  
        properties.BeginningSequenceNumber + 1);
```

```
    }
```

```
}
```

// Start retrieving events from the event hub and print to the console

```
int retrievedCount = 0;

await foreach (PartitionEvent partitionEvent in
consumerClient.ReadEventsAsync(startReadingAtEarliestEvent: true))
{
    if (partitionEvent.Data != null)
    {
        string body = Encoding.UTF8.GetString(partitionEvent.Data.Body.ToArray());
        Console.WriteLine($"Retrieved event: {body}");
        retrievedCount++;
        if (retrievedCount >= totalEventCount)
        {
            Console.WriteLine("Done retrieving events. Press Enter to exit...");
            Console.ReadLine();
            return;
        }
    }
}
```

4. Press **ctrl+s** to save the file, then **ctrl+q** to exit the editor.

Sign into Azure and run the app

1. In the cloud shell command-line pane, enter the following command to sign into Azure.

TypeCopy

az login

You must sign into Azure - even though the cloud shell session is already authenticated.

Note: In most scenarios, just using `az login` will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the `--tenant` parameter. See [Sign into Azure interactively using Azure CLI](#) for details.

2. Start the application by running the following command:

```
TypeCopy
```

```
dotnet run
```

After a few seconds you should see output similar to the following example:

```
TypeCopy
```

```
A batch of 3 events has been published.
```

```
Press Enter to retrieve and print the events...
```

```
Retrieving all events from the hub...
```

```
Retrieved event: Event 4
```

```
Retrieved event: Event 96
```

```
Retrieved event: Event 74
```

```
Done retrieving events. Press Enter to exit...
```

The application always sends three events to the hub, but it retrieves all events in the hub. If you run the application multiple times an increasing number of events are retrieved. The random numbers used for event creation help you identify different events.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Navigate to the resource group you created and view the contents of the resources used in this exercise.
3. On the toolbar, select **Delete resource group**.
4. Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.